



Vriddhi Nexus Bus Booking Platform

Complete but concise platform overview for client review

Current phase

The product is mock-ready for client review. It uses mock bus inventory, mock seat holds, mock payment status, mock invoices, scheduled-route tracking, and mock support/refund queues. Real supplier APIs, live payment capture, live GPS, and live external data are not connected yet.

Platform Summary

Vriddhi Nexus is a web-based bus booking platform for customers, admins, and travel agents. It supports route search, seat selection with prices, passenger details, mock payment completion, ticket and invoice generation, role dashboards, reporting, operational controls, support, refunds, and launch-readiness architecture.

Customer

Search routes, compare buses, select seats, book tickets, view invoices, track journey status, save travellers, use support, and review rewards.

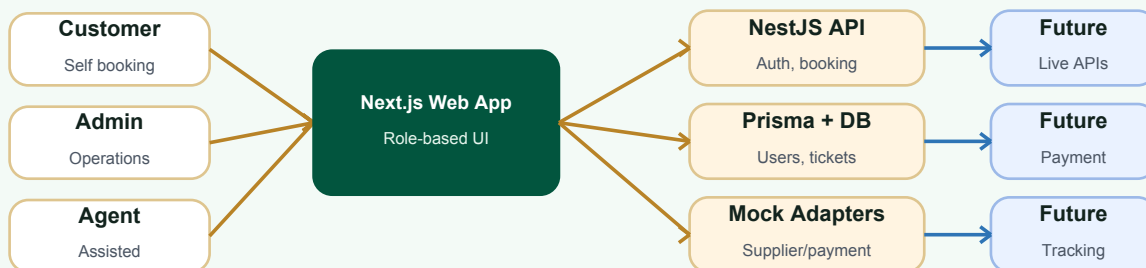
Admin

Manage bookings, seat pricing, invoice generation, bulk booking upload, users, agents, customers, refunds, support desk, reports, trust signals, CMS, settings, and monitoring.

Travel Agent

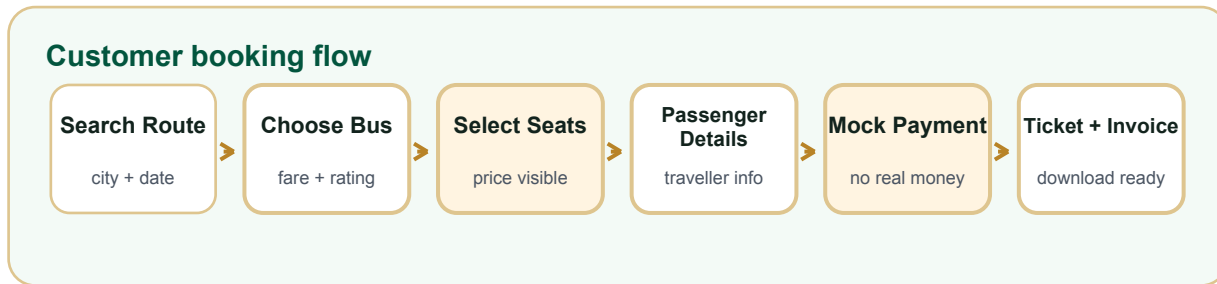
Perform assisted booking, manage agent customers, view bookings, reports, notifications, profile, settings, and help workflows.

High-level platform structure



Customer Booking Experience

The customer side is designed to stay simple: search first, compare clearly, choose seats, enter passenger details, confirm through a mock payment flow, and receive ticket plus invoice records.



Search and filters

From city, to city, journey date, timing windows, bus type, fare range, seat availability, operator, rating, amenities, and tracking filter.

Seat layout

Upper/lower deck, seat prices, selected seats, booked seats, female/male indicators, and admin-controlled seat fare data.

After booking

Booking history, upcoming trips, past trips, cancelled trips, tickets, invoices, notifications, tracking preview, saved travellers, support, and refund status.

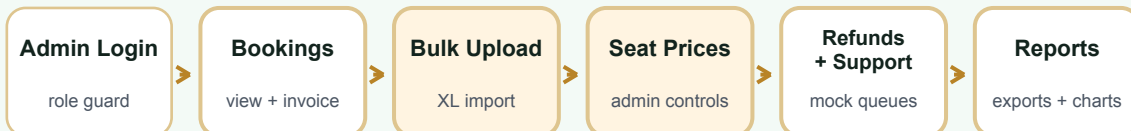
Customer pages included

Area	Included pages / functions
Booking	Search, bus results, seat layout, passenger details, booking review, confirmation, failed booking, ticket, download ticket.
Account	Login, register, forgot/reset password, verify email, profile, change password, settings, session expired.
Dashboard	Customer dashboard, bookings, invoices, tickets, notifications, saved travellers, wallet/rewards, tracking, support.

Admin And Agent Operations

The back-office is role-based and separates operational work from customer booking. Customers cannot access the admin dashboard. Admins and agents see their own navigation, dashboards, and workflows.

Admin operations flow



Module	What is included
Bookings and invoices	Booking list, details, ticket actions, invoice generation, invoice visibility, cancellation and reschedule placeholders.
Bulk booking upload	Admin upload area for XL/Excel bulk booking data with invoice generation path.
Seat and fare controls	Seat layout/price data can be managed from admin-side inventory workflows before customer booking.
Users and access	Users, customers, agents, roles, permissions, audit logs, activity logs, profile, and admin-only route protection.
Operations	Refund desk, support desk, notifications, email templates, coupons, offers, CMS, platform settings, feature flags.
Reports and monitoring	Analytics, reports, system monitoring, supplier configuration, integration health placeholders, queues, metrics, and readiness views.

Agent portal

Quick booking, bookings, customers, reports, notifications, profile, settings, and help.

Invoice flow

Customer booking and admin bulk bookings create invoice-ready records in the mock workflow.

Role safety

Admin, agent, and customer workspaces are separated with route guards and dashboard redirects.

Technical And Launch Readiness

The codebase is a pnpm/Turborepo monorepo with a Next.js frontend, NestJS API, Prisma/PostgreSQL data model, Redis/BullMQ readiness, reusable UI package, shared types, supplier SDK contracts, Docker, CI, and deployment documentation.

Frontend

Next.js 15, React 19, TypeScript, Tailwind CSS, reusable UI components, role dashboards, forms, charts, responsive layouts.

Backend

NestJS modules for auth, booking, search, seats, tickets, payment placeholders, admin, user, agent, reports, notifications, tracking, and monitoring.

Data and infra

Prisma schema, PostgreSQL readiness, Redis/cache/queue readiness, Docker, GitHub Actions, environment templates, health and metrics endpoints.

Integration status

Area	Current status	Future connection
Bus inventory	Mock supplier adapter active	Connect real supplier APIs such as BCI, RedBus, AbhiBus, TBO, or custom API.
Payments	Mock payment status only	Connect selected gateway and webhook verification before real capture/refund.
Tracking	Scheduled-route preview	Connect live GPS/tracking provider after supplier onboarding.
Messaging	Email/SMS/WhatsApp/in-app templates represented	Connect real SMTP, SMS, WhatsApp, and push providers.
Storage	PDF/object storage placeholders	Connect production object storage for ticket/invoice files.

What the client can review now

1	Customer booking journey with clean UI and mock data.
2	Seat price visibility and admin-side readiness for fare updates.
3	Ticket, invoice, booking history, cancellation, refund, and support surfaces.
4	Admin, customer, and agent workspaces with role-based access separation.
5	Deployment readiness pieces: health checks, metrics, Docker, CI, environment templates, and mock integration boundaries.

Recommended next steps

Run client UAT with sample bookings, approve the final UI and workflows, select supplier/payment/tracking providers, connect production services, run security and load checks, then launch with monitoring and rollback plan.